

NBA Workforce Planner & Stat Forecaster

Project Overview

NBA player performance engine applying Workforce Management (WFM) principles to sports analytics. Features a 240-minute capacity cap, waterfall trend weighting, and system-latency normalization. Achieved 60.87% accuracy (71% on unders) in 2026 backtests. Transitioning logic to MLB "Machine Learning Ball" for the 2026 season.

By treating an NBA roster as a high-volume "Workforce" with finite capacity and variable demand, the model identifies market inefficiencies that traditional regression-based models often overlook. This project demonstrates the power of cross-domain logic—applying 20 years of Fortune 50 operational strategy to the high-variance world of sports data.

The "Workforce" Logic: Queueing Theory on the Court

Unlike traditional models that look at players in a vacuum, this model treats an NBA team as a system with a fixed capacity and strict operational constraints:

- **Strict 240-Minute Cap (Resource Allocation):** In WFM, you cannot schedule more staff than you have desks. In the NBA, there are only 240 minutes available per regulation game. The model normalizes player minutes to ensure team totals reflect a 48-minute, 5-position game, preventing the "over-projection" common in aggregate-based models.
- **Waterfall Trend Weighting (Service Level Agreement):** The engine utilizes a "Waterfall" methodology—prioritizing recent performance (recency bias) to capture "hot streaks" while maintaining a season-long baseline to ensure statistical durability. This mirrors how call center volume is forecasted by weighing the last 3 weeks heavier than the last 3 months.
- **Opponent-Strength Normalization (System Latency):** Forecasts are adjusted based on the defensive efficiency and pace of the opposing "system." If a high-volume scorer meets a high-efficiency defense, the model calculates the "shrinkage" in expected output.

Performance Metrics & Validation

The model was validated against a full dataset of ~18,000 matched player-stat rows from Jan 12, 2026, to Feb 2, 2026.

- **PTS MAE (Mean Absolute Error):** 5.15. This is highly competitive with top-tier commercial DFS models, which typically range between 5.0 and 7.0.

- **Under Bias (The Profit Edge):** The model achieved a **71% hit rate on "Unders."** By accurately modeling system capacity (minutes/touches), it identifies when bookmakers over-project volume due to public sentiment or "star power."
- **Out-of-Sample Validation:** All results were generated as "Forward-Looking" forecasts (produced daily before 4:00 PM Central) and compared against real-world betting lines and post-game results to eliminate lookahead bias.

Future Roadmap: MLB "Machine Learning Ball"

The success of the NBA pilot has led to the development of **Machine Learning Ball (MLB)** for the 2026 season. This next evolution applies more advanced WFM and Engineering concepts:

- **Hitting as Throughput:** Modeling hitters as "Service Agents" and baserunners as "Incoming Calls."
- **The Queueing Advantage:** Using M/M/c and Monte Carlo simulations to predict "System Latency" (Runners Left on Base).
- **Pitching as System Failure:** Utilizing **Erlang-C** logic to predict the probability of a "System Blowout" based on bullpen utilization rates and fatigue factors.

Tech Stack

- **Language:** Python (Pandas, NumPy for vectorized data manipulation)
- **Data Pipeline:** BallDon'tLie API (GOAT Tier) for high-frequency, real-time stat ingestion.
- **Environment:** Google Colab / LLMOps for rapid feature prototyping and automated daily execution.
- **Architecture:** Modular "Lean AI" stack designed for scalability and low-latency inference.

NBA Code Summary & Execution Logic

The core of this repository is a specialized WorkforcePlanner class that orchestrates the data pipeline. Here is the technical workflow:

1. **Data Ingestion:** The script uses a multi-threaded BallDon'tLie wrapper to pull 15-game and full-season splits simultaneously, managing API rate limits via a custom retry-logic decorator.
2. **The "Shrinkage" Calculation:** It calculates a player's "PPM" (Production Per Minute) across six categories. Instead of a simple average, it applies a **Waterfall Weighting** (40% recent / 60% season) to account for role changes and "hot hands."
3. **Capacity Normalization:** The most critical step. The script sums the projected minutes for all active players on a team. If the total exceeds **240 minutes**, it applies a scaling factor to redistribute the "supply" of minutes realistically across the "workforce" (the roster).
4. **Opponent Modifiers:** The model fetches defensive efficiency metrics for the opponent, applying a "Service Difficulty" scalar to the player's baseline stats based on defensive rank and league-average pace.

5. **Output & Reshaping:** Finally, the script reshapes the data into a long-form format suitable for Google Sheets integration, allowing for real-time comparison against betting lines.

Author: Chad Corwin

Title: Associate Director | Operational Strategy & AI Innovation

Contact: corwincarthur@gmail.com | [LinkedIn Profile](#)

X: @MLBallForecast